```
$ make
gcc -Wall -c main.c
gcc -o programa *.o
>>> OK ✓
```

ESCOM · IPN

Uso y Documentación de Makefiles

Alumno: MORAN VAQUERO MARCOS

Boleta: 2023630138

Materia: Sistemas Embebidos e IoT

Carrera: Ingeniería en Sistemas Computacionales

Escuela: ESCOM – IPN

Abril 2025

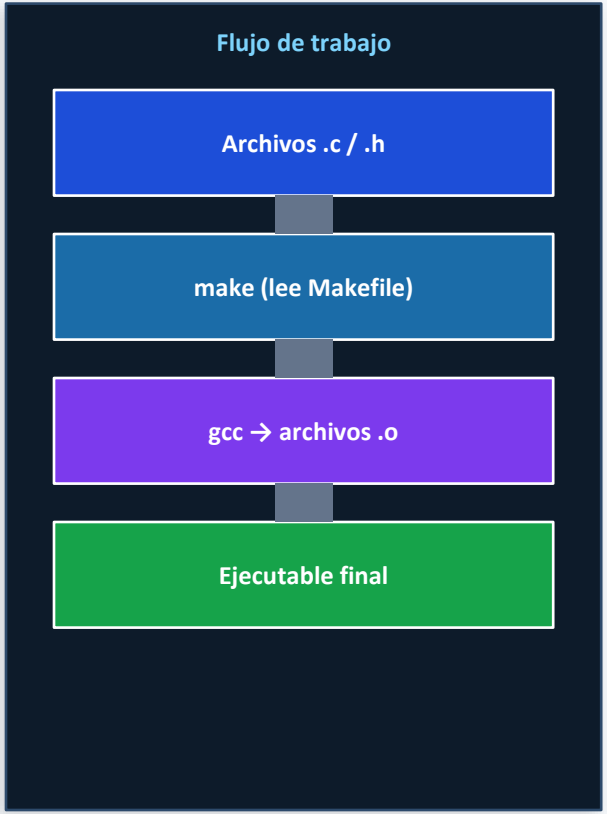
¿Qué es un Makefile?

Definición

Un Makefile es un archivo de texto que contiene un conjunto de reglas utilizadas por la herramienta make para automatizar la compilación de programas. Permite definir dependencias entre archivos y ejecutar comandos de forma ordenada y eficiente.

Ventajas principales

- Automatiza la compilación de múltiples archivos fuente
- Recompila sólo lo que cambió (compilación incremental)
- Estandariza los comandos en equipos de desarrollo
- Reduce errores al eliminar comandos manuales repetitivos
- Facilita el mantenimiento de proyectos grandes



Estructura básica de un Makefile

Makefile

```
# Variables
CC      = gcc
CFLAGS  = -Wall -Wextra -g
TARGET  = programa
SRCS    = main.c funciones.c
OBJS    = $(SRCS:.c=.o)

# Target por defecto
all: $(TARGET)

# Enlazado
$(TARGET): $(OBJS)
                $(CC) $(CFLAGS) -o $@ $^

# Compilación por patrón
%.o: %.c
                $(CC) $(CFLAGS) -c $< -o $@

clean:
                rm -f $(OBJS) $(TARGET)

.PHONY: all clean
```

Variables

Definen compilador, flags y archivos. Reutilizables en todo el Makefile.

Regla de patrón %.o: %.c

Compila automáticamente cualquier .c a .o sin listarlos uno a uno.

.PHONY

Declara que clean y all no son archivos reales del sistema.

Ejecución del ejemplo del repositorio

PASO 1 — Compilar con make

```
$ make
gcc -Wall -Wextra -g -c main.c -o main.o
gcc -Wall -Wextra -g -c funciones.c -o funciones.o
gcc -Wall -Wextra -g -o programa main.o funciones.o
```

PASO 2 — Ejecutar el programa

```
$ ./programa
=== Programa de ejemplo - Makefiles ===
Hola desde ESCOM!
Resultado de 10 + 25 = 35
```

PASO 3 — Limpiar archivos generados

```
$ make clean
rm -f main.o funciones.o programa
```

¿Qué observamos en la salida?

Make imprime cada comando antes de ejecutarlo, así puedes ver exactamente qué ocurre en cada paso de la compilación.

Los archivos se compilan en el orden correcto: primero los .o de cada módulo y luego el enlazado final.

Si modificas solo un archivo y vuelves a ejecutar make, únicamente ese módulo se recompila. Los demás .o se reutilizan.

make clean elimina TODOS los archivos generados. Es necesario antes de una compilación limpia desde cero.

Proyecto propio – Conversor y Analizador de Temperaturas

Programa modular en C que recibe lecturas de temperatura en Celsius (simulando un sensor IoT) y las convierte a Fahrenheit y Kelvin, calculando además estadísticas básicas del conjunto de datos.

Estructura del repositorio

```
mi_proyecto/
├── Makefile
├── .gitignore
├── README.md
└── src/
    ├── main.c
    ├── conversor.c
    ├── conversor.h
    ├── estadisticas.c
    └── estadisticas.h
```

main.c

Punto de entrada. Itera sobre las lecturas del sensor e imprime la tabla de conversiones y estadísticas.

conversor.c/.h

Implementa las fórmulas de conversión: Celsius → Fahrenheit y Celsius → Kelvin.

estadisticas.c/.h

Calcula mínimo, máximo y promedio del arreglo de lecturas del sensor.

Makefile del proyecto propio – Análisis

Makefile – mi_proyecto

```
CC      = gcc
CFLAGS  = -Wall -Wextra -Wpedantic -std=c11 -g
TARGET  = temp_analyzer
SRCS    = src/main.c src/conversor.c src/estadisticas.c
OBJS    = $(SRCS:src/%.c=build/%.o)
BUILD_DIR = build

all: $(BUILD_DIR) $(TARGET)

$(BUILD_DIR):
                mkdir -p $(BUILD_DIR)

$(TARGET): $(OBJS)
                $(CC) $(CFLAGS) -o $@ $^
                @echo ">>> Compilacion exitosa"

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.c
                $(CC) $(CFLAGS) -c $< -o $@

clean run help
.PHONY: all clean run help
```

Mejoras respecto al ejemplo de clase

Directorio build/ separado

Los archivos .o se guardan en build/, manteniendo la carpeta src/ limpia con solo código fuente.

Sustitución de texto avanzada

OBJS usa $(SRCS:src/%.c=build/%.o) para transformar rutas automáticamente sin escribirlas una a una.

Estándar C11 + Wpedantic

Fuerza el cumplimiento estricto del estándar C11, detectando más advertencias durante la compilación.

Target help

Ejecutar make help imprime todos los comandos disponibles, útil para equipos o revisores del repositorio.

Ejecución del proyecto propio – Resultados

$ make run → Compilar y ejecutar

```
$ make run
mkdir -p build
gcc -Wall -Wextra -Wpedantic -std=c11 -g \
  -c src/main.c -o build/main.o
gcc ... -c src/conversor.c -o build/conversor.o
gcc ... -c src/estadisticas.c -o build/estadisticas.o
gcc ... -o temp_analyzer build/*.o
>>> Compilacion exitosa: temp_analyzer
./temp_analyzer
========================================
  Conversor y Analizador de Temperaturas
  ESCOM – Sistemas Embebidos e IoT
========================================
Celsius   Fahrenheit  Kelvin
22.50     72.50       295.65
36.70     98.06       309.85
100.00    212.00      373.15
...
Minima  : -40.00 C
Maxima  : 100.00 C
Promedio:  25.93 C
```

.gitignore

```
build/          ← carpeta de .o
temp_analyzer   ← ejecutable
*.o             ← objetos sueltos
*.swp  *~  .DS_Store
```

Compilación incremental

Si solo modificas conversor.c y vuelves a ejecutar make, únicamente ese archivo se recompila. estadisticas.o y main.o ya existen y están actualizados, así que Make los reutiliza.

Archivo .gitignore y organización del repositorio

¿Por qué es importante?

Un repositorio limpio solo contiene los archivos fuente necesarios para reproducir el proyecto. Los archivos generados (ejecutables, .o) cambian con cada compilación y no deben subirse a Git.

.gitignore

```
# Archivos objeto (compilación)
build/
*.o

# Ejecutable compilado
temp_analyzer

# Volcados de memoria
core  core.*  *.dSYM/

# Temporales de editores
*.swp  *.swo  *~
.DS_Store  Thumbs.db
.vscode/  .idea/
```

Sin .gitignore

- main.c
- funciones.c
- funciones.h
- main.o ✗
- funciones.o ✗
- programa ✗
- .DS_Store ✗
- core ✗

Con .gitignore

- main.c ✓
- funciones.c ✓
- funciones.h ✓
- Makefile ✓
- README.md ✓
- .gitignore ✓

Conclusiones y aprendizajes

01

Automatización real

Un solo comando (make) reemplaza decenas de comandos de GCC escritos a mano, eliminando errores por omisión.

02

Compilación incremental

Make detecta qué archivos cambiaron y recompila únicamente esos módulos, ahorrando tiempo en proyectos grandes.

03

Modularidad y mantenimiento

Dividir el código en módulos (.c separados) hace que el Makefile escale sin necesidad de grandes cambios.

04

Repositorio limpio

El .gitignore garantiza que el repositorio solo contenga lo necesario para reproducir el proyecto.

05

Estándar en la industria

Makefiles son utilizados en proyectos de sistemas embebidos, kernels y firmware en entornos profesionales reales.

MORAN VAQUERO MARCOS
2023630138